

lab4

March 23, 2026

1 Głębokie Uczenie w Praktyce

1.1 Laboratorium 4

1.1.1 Mateusz Horczak

```
[12]: import os
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
from tensorflow.keras import *
```

Ćwiczenie 1. Konwolucyjna sieć do rozpoznawania cyfr z bazy MNIST

```
[13]: model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_8 (Conv2D)	(None, 11, 11, 64)	18,496

max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_9 (Conv2D)	(None, 3, 3, 64)	36,928
flatten_2 (Flatten)	(None, 576)	0
dense_4 (Dense)	(None, 64)	36,928
dense_5 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)

Trainable params: 93,322 (364.54 KB)

Non-trainable params: 0 (0.00 B)

```
[14]: (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype('float32') / 255
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

```
[15]: model.compile(optimizer='rmsprop',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
model.fit(train_images, train_labels, epochs=5, batch_size=64)

test_loss, test_acc = model.evaluate(test_images, test_labels)
print("test_acc: ", test_acc)
```

```
Epoch 1/5
938/938          20s 20ms/step -
accuracy: 0.9445 - loss: 0.1781
Epoch 2/5
938/938          20s 22ms/step -
accuracy: 0.9846 - loss: 0.0484
Epoch 3/5
938/938          20s 21ms/step -
accuracy: 0.9901 - loss: 0.0325
Epoch 4/5
938/938          20s 22ms/step -
accuracy: 0.9924 - loss: 0.0247
Epoch 5/5
```

```
938/938          20s 21ms/step -
accuracy: 0.9940 - loss: 0.0197
313/313          2s 4ms/step -
accuracy: 0.9919 - loss: 0.0252
test_acc: 0.9919000267982483
```

Ćwiczenie 2. Sieć do rozpoznawania zdjęć psów i kotów - Dogs-Cats – pobieranie danych

```
[16]: train_dir = 'train-test-valid/train'
validation_dir = 'train-test-valid/validation'
test_dir = 'train-test-valid/test'

train_cats_dir = os.path.join(train_dir, 'cats')
train_dogs_dir = os.path.join(train_dir, 'dogs')
validation_cats_dir = os.path.join(validation_dir, 'cats')
validation_dogs_dir = os.path.join(validation_dir, 'dogs')
test_cats_dir = os.path.join(test_dir, 'cats')
test_dogs_dir = os.path.join(test_dir, 'dogs')

print("liczba obrazow treningowych kotow", len(os.listdir(train_cats_dir)))
print('liczba obrazów treningowych psow: ', len(os.listdir(train_dogs_dir)))
print('liczba obrazów walidacyjnych kotow: ', len(os.
↳listdir(validation_cats_dir)))
print('liczba obrazów walidacyjnych psow: ', len(os.
↳listdir(validation_dogs_dir)))
print('liczba obrazów testowych kotow:', len(os.listdir(test_cats_dir)))
print('liczba obrazów testowych psow: ', len(os.listdir(test_dogs_dir)))
```

```
liczba obrazow treningowych kotow 1000
liczba obrazów treningowych psow: 1000
liczba obrazów walidacyjnych kotow: 500
liczba obrazów walidacyjnych psow: 500
liczba obrazów testowych kotow: 500
liczba obrazów testowych psow: 500
```

Ćwiczenie 3. Dogs-Cats - budowa sieci neuronowej

```
[17]: model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
```

```

model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(learning_rate=1e-4),
              metrics=['acc'])
model.summary()

```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_8 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_11 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_9 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_12 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_10 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_13 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_11 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_3 (Flatten)	(None, 6272)	0
dense_6 (Dense)	(None, 512)	3,211,776
dense_7 (Dense)	(None, 1)	513

Total params: 3,453,121 (13.17 MB)

Trainable params: 3,453,121 (13.17 MB)

Non-trainable params: 0 (0.00 B)

Ćwiczenie 4. Dogs-Cats - wstępna obróbka danych

```

[ ]: train_datagen = ImageDataGenerator(rescale=1 / 255)
     test_datagen = ImageDataGenerator(rescale=1 / 255)

```

```

train_generator = train_datagen.flow_from_directory(
    train_dir, # katalog docelowy
    target_size=(150, 150), # przeskalowanie do 150x150
    batch_size=20,
    class_mode='binary')
validation_generator = test_datagen.flow_from_directory(validation_dir,
    ↪target_size=(150, 150), batch_size=20,
                                                    class_mode='binary')

```

```

[19]: for data_batch, labels_batch in train_generator:
    print('kształt danych wsadowych:', data_batch.shape)
    print('kształt etykiet danych wsadowych:', labels_batch.shape)
    break

```

kształt danych wsadowych: (20, 150, 150, 3)
 kształt etykiet danych wsadowych: (20,)

Ćwiczenie 5. Dogs-Cats – trening sieci, wykresy straty i dokładności

```

[20]: history = model.fit(train_generator, steps_per_epoch=100, epochs=30,
    validation_data=validation_generator, validation_steps=50)

```

```

Epoch 1/30
100/100          51s 501ms/step -
acc: 0.5290 - loss: 0.6935 - val_acc: 0.6090 - val_loss: 0.6844
Epoch 2/30
100/100          49s 493ms/step -
acc: 0.5875 - loss: 0.6777 - val_acc: 0.5000 - val_loss: 0.7225
Epoch 3/30
100/100          44s 440ms/step -
acc: 0.6370 - loss: 0.6525 - val_acc: 0.6380 - val_loss: 0.6423
Epoch 4/30
100/100          45s 450ms/step -
acc: 0.6550 - loss: 0.6258 - val_acc: 0.6420 - val_loss: 0.6363
Epoch 5/30
100/100          45s 445ms/step -
acc: 0.6890 - loss: 0.5884 - val_acc: 0.6590 - val_loss: 0.6069
Epoch 6/30
100/100          45s 453ms/step -
acc: 0.7055 - loss: 0.5595 - val_acc: 0.6730 - val_loss: 0.6020
Epoch 7/30
100/100          50s 500ms/step -
acc: 0.7200 - loss: 0.5352 - val_acc: 0.6820 - val_loss: 0.5844
Epoch 8/30
100/100          47s 475ms/step -
acc: 0.7340 - loss: 0.5153 - val_acc: 0.6960 - val_loss: 0.5757
Epoch 9/30
100/100          44s 437ms/step -
acc: 0.7640 - loss: 0.4891 - val_acc: 0.6960 - val_loss: 0.5653

```

Epoch 10/30
100/100 49s 488ms/step -
acc: 0.7785 - loss: 0.4642 - val_acc: 0.7060 - val_loss: 0.5667
Epoch 11/30
100/100 79s 462ms/step -
acc: 0.8025 - loss: 0.4355 - val_acc: 0.6940 - val_loss: 0.5896
Epoch 12/30
100/100 77s 409ms/step -
acc: 0.8110 - loss: 0.4126 - val_acc: 0.7230 - val_loss: 0.5521
Epoch 13/30
100/100 44s 442ms/step -
acc: 0.8315 - loss: 0.3875 - val_acc: 0.7150 - val_loss: 0.5587
Epoch 14/30
100/100 78s 408ms/step -
acc: 0.8340 - loss: 0.3648 - val_acc: 0.7240 - val_loss: 0.5619
Epoch 15/30
100/100 44s 437ms/step -
acc: 0.8595 - loss: 0.3345 - val_acc: 0.7110 - val_loss: 0.5687
Epoch 16/30
100/100 44s 439ms/step -
acc: 0.8710 - loss: 0.3200 - val_acc: 0.7220 - val_loss: 0.5687
Epoch 17/30
100/100 44s 438ms/step -
acc: 0.8805 - loss: 0.2962 - val_acc: 0.7140 - val_loss: 0.5937
Epoch 18/30
100/100 44s 437ms/step -
acc: 0.8970 - loss: 0.2572 - val_acc: 0.6990 - val_loss: 0.6563
Epoch 19/30
100/100 44s 444ms/step -
acc: 0.9130 - loss: 0.2289 - val_acc: 0.6650 - val_loss: 0.7812
Epoch 20/30
100/100 79s 416ms/step -
acc: 0.9095 - loss: 0.2226 - val_acc: 0.6640 - val_loss: 1.1537
Epoch 21/30
100/100 44s 443ms/step -
acc: 0.9320 - loss: 0.1944 - val_acc: 0.7190 - val_loss: 0.6559
Epoch 22/30
100/100 79s 418ms/step -
acc: 0.9455 - loss: 0.1664 - val_acc: 0.7200 - val_loss: 0.6645
Epoch 23/30
100/100 47s 467ms/step -
acc: 0.9440 - loss: 0.1575 - val_acc: 0.6900 - val_loss: 0.8670
Epoch 24/30
100/100 47s 468ms/step -
acc: 0.9565 - loss: 0.1379 - val_acc: 0.7190 - val_loss: 0.7732
Epoch 25/30
100/100 84s 489ms/step -
acc: 0.9625 - loss: 0.1105 - val_acc: 0.7110 - val_loss: 0.8897

```
Epoch 26/30
100/100          46s 459ms/step -
acc: 0.9655 - loss: 0.1020 - val_acc: 0.7210 - val_loss: 0.8042
Epoch 27/30
100/100          52s 517ms/step -
acc: 0.9750 - loss: 0.0834 - val_acc: 0.7190 - val_loss: 0.9195
Epoch 28/30
100/100          47s 469ms/step -
acc: 0.9775 - loss: 0.0752 - val_acc: 0.7120 - val_loss: 0.8958
Epoch 29/30
100/100          49s 489ms/step -
acc: 0.9775 - loss: 0.0664 - val_acc: 0.6890 - val_loss: 1.0786
Epoch 30/30
100/100          53s 533ms/step -
acc: 0.9870 - loss: 0.0518 - val_acc: 0.7200 - val_loss: 0.9989
```

```
[21]: acc = history.history['acc']
      val_acc = history.history['val_acc']
      loss = history.history['loss']
      val_loss = history.history['val_loss']
      epochs = range(len(acc))
      plt.plot(epochs, acc, 'bo', label='Dokladnosc trenowania')
      plt.plot(epochs, val_acc, 'b', label='Dokladnosc walidacji')
      plt.title('Dokladnosc trenowania i walidacji')
      plt.legend()
      plt.figure()
      plt.plot(epochs, loss, 'bo', label='Strata trenowania')
      plt.plot(epochs, val_loss, 'b', label='Strata walidacji')
      plt.title('Strata trenowania i walidacji')
      plt.legend()
      plt.show()
```



